

CoStream: Composing Simple Behaviors for Generalizable Complex Manipulation

Haonan Chen^{*1,2} Yuxiang Ma^{*3}
Stephen Tian² Xiaoshen Han¹ Wenlong Huang² Feiyang Wu¹
Yunzhu Li⁴ Jiajun Wu² Edward H. Adelson³ Yilun Du¹

¹Harvard University ²Stanford University

³Massachusetts Institute of Technology ⁴Columbia University

Abstract: Long-horizon, contact-rich complex manipulation tasks, such as seating a GPU into a PCIe slot, demand both millimeter high precision and out-of-the-box generalization to new tasks. Existing paradigms struggle to satisfy both: classical pipelines use brittle, task-specific interfaces to achieve high-precision control but require costly pipeline redesigns to adapt to new tasks, whereas monolithic end-to-end policies provide better generalization but lack high precision on complex, out-of-distribution tasks unless retrained with new data. Both paradigms share an implicit assumption: once a manipulation capability is acquired, it must be deployed as a rigid pipeline or monolithic whole, rather than being freely decomposed and recomposed. In this paper, we show that complex manipulation capabilities can emerge naturally from the composition of simple, independent behaviors. Rather than deploying a monolithic policy or a rigid pipeline, we propose **CoStream**, a framework orchestrating foundation models and diverse sensing modalities into multiple composable core behaviors: a semantic behavior extracting spatial constraints via foundation models; a predictive behavior forecasting trajectories by tracking keypoints in imagined videos; and a reactive behavior providing high-frequency tactile and force corrections. On a shared $SE(3)$ interface, these outputs compose by right-multiplication into a single pose command at each control step, executed by a compliant controller. We demonstrate **CoStream** on 8 real-world tasks spanning everyday manipulation and precision assembly, with the strongest gains in contact-rich assembly and object transfer, and show robust recovery from manual perturbations during execution. Website: <https://costream-simple.github.io>

Keywords: Behavior Composition, $SE(3)$ Control, Tactile Manipulation

1 Introduction

Inserting a GPU into a PCIe slot leaves less than a millimeter of clearance. A slight angular error can make the card catch on the slot rim and jam before seating. If the card slips inside the gripper mid-insertion, the entire trajectory jams. Long-horizon, contact-rich manipulation like GPU insertion requires a robot to understand the semantic geometry of a motherboard, predict a collision-free path, and physically react to sub-millimeter force deviations in real time.

To address these challenges, current research generally falls into two paradigms. The first paradigm employs classical, modular pipelines that explicitly decouple perception, state estimation, planning, and control. These pipelines are inherently rigid; adapting them to new tasks requires bespoke engineering and the manual redesign of perception and state estimation modules [1, 2, 3]. The second relies on large monolithic policies, such as vision-language-action (VLA) models. These

^{*}Equal contribution.

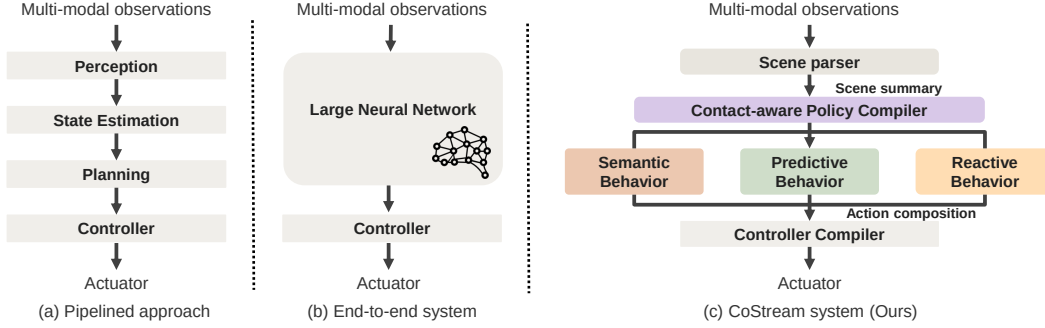


Figure 1: **CoStream: Composing Simple Behaviors for Complex Contact-Rich Manipulation.** **CoStream** composes multiple sensor-grounded behaviors into a single end-effector command. A semantic behavior parses instructions with an LLM and a VLM into geometric constraints. A predictive behavior extracts a 3D reference trajectory from a video world model. A reactive behavior closes a high-rate loop from tactile and force feedback. The behaviors share an $SE(3)$ interface and compose by right-multiplication into one pose command at every control step, which a compliant controller executes while regulating contact force.

models attempt to absorb semantic reasoning and high-frequency contact dynamics into a single network to solve tasks like GPU insertion directly. However, these end-to-end approaches are highly data-hungry, requiring extensive new human demonstrations to generalize to every novel task [4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]. Both paradigms share the assumption that complex tasks require a complex and indivisible control system.

Behavior-based robotics, a pioneering framework from the 1980s, argues that *complex behavior need not arise from a single complex controller; it can instead emerge from composing simple sensory behaviors through interaction with a complex environment* [16, 17]. Inspired by this idea, we show that complex manipulation can be achieved by composing multiple simple, independent action components. Much like human motor control, breaking actions down ensures that each component remains simple. Consequently, whether a behavior is powered by a foundation model, a specialized neural network, or classical engineering, it becomes significantly easier to develop and naturally accommodates diverse sensor modalities. Building on this principle, **CoStream** establishes a common $SE(3)$ task-space interface across three behaviors: a semantic behavior that produces object-centric geometric goals, a predictive behavior that generates nominal motion from imagined video, and a reactive behavior that produces high-frequency tactile and force contact corrections. Unlike a conventional modular pipeline, **CoStream** does not pass a completed plan through a sequence of fixed interfaces. Instead, these behaviors remain independent, operate at their natural rates, and compose by right-multiplication into a single pose command at every control step, which a compliant controller executes.

Our contributions are threefold. First, we introduce a behavior composition interface for contact-rich manipulation that separates semantic grounding, predictive motion generation, and tactile and force reaction into independent behaviors. Second, we propose a shared $SE(3)$ task space representation that allows each behavior to run at its natural rate and contribute only the component it is reliable for: a task frame, a nominal motion prior, or a contact correction. Third, we validate **CoStream** on eight real world tasks, showing high precision assembly, manual perturbation recovery, and transfer to everyday manipulation without retraining a policy for each task.

2 Related Works

Foundation Models for Robotics. Large Language Models (LLMs) and Vision-Language Models (VLMs) have enabled robots to interpret language, reason over long-horizon tasks, and generate executable plans [18, 4, 19, 20, 21, 22]. Recent methods improve spatial precision by grounding robot actions in visual observations, using value maps [23], keypoint or affordance constraints [24, 25, 26], iterative visual prompting [27], or test-time simulation [28]. However, these approaches still rely primarily on vision or predictive simulation, and therefore lack the real-time tactile and force feedback needed for contact-rich manipulation.

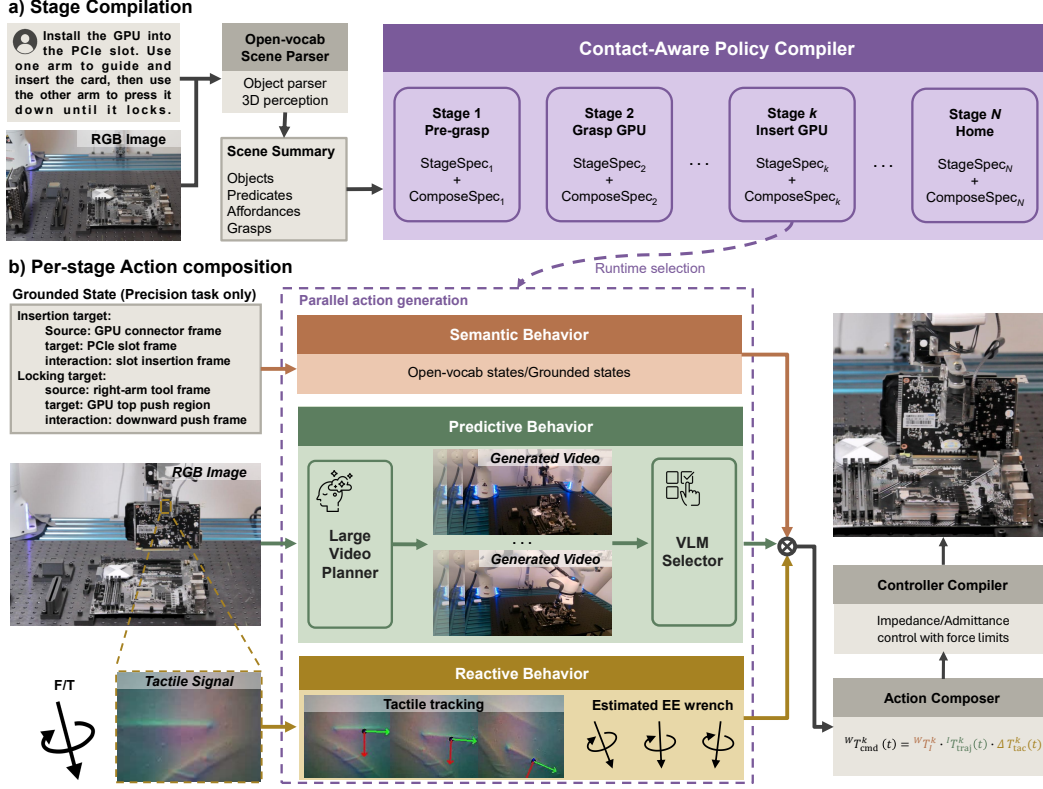


Figure 2: **CoStream Architecture**. A scene parser converts language goals and observations into a Scene Summary, and a contact-aware policy compiler expands it into Stage and Composition Specifications. For each stage, the semantic behavior produces a task-frame anchor, the predictive behavior produces a nominal motion prior in that frame, and the reactive behavior produces a tactile residual and guard events. The action composer forms $W_{T_{cmd}}^k(t)$ by right-multiplying the anchor, nominal motion, and tactile residual, while the controller compiler instantiates impedance/admittance parameters, force limits, guards, and recovery behavior for compliant execution.

Tactile and Force Sensing in Robotics. Touch provides physical interaction cues that vision cannot capture, especially in contact-rich tasks such as cable routing, packing, and needle threading [29, 30, 31, 32]. Existing tactile methods are either analytic, using tactile signals as constraints or triggers for controllers [33, 34], or data-driven, learning skills through reinforcement learning [35, 36], imitation learning [37, 38, 39, 40, 41], or tactile-informed MPC [42, 32]. Analytic methods often require task-specific models, while data-driven methods are data-hungry and tend to specialize to the training task. In contrast, our approach uses tactile feedback to adapt object-centric behaviors online, enabling real-time adaptation and zero-shot generalization without task-specific training.

Compositional Modeling for Robotics. Compositionality is central to generalizable robot behavior. Traditional modular approaches, including Task and Motion Planning (TAMP) [43] and layered control architectures [17], provide structure but require hand-specified interfaces and constraints. Monolithic end-to-end policies [44, 4, 45, 46] reduce manual engineering but learn entangled mappings that struggle with compositional generalization and require retraining. Recent work composes heterogeneous, factorized, or multimodal policies [47, 48, 41], while another line composes foundation models sequentially from text to visual plans and actions [49, 50]. Our method instead composes semantic, visual, and reactive streams in parallel, avoiding both the manual engineering of TAMP and the retraining requirements of end-to-end policies.

3 Methodology

CoStream separates long-horizon contact-rich manipulation into two levels: *stage compilation* and *per-stage action composition* (Fig. 2). An open-vocabulary scene parser converts a language instruction and RGB-D observations into a *Scene Summary*, which a contact-aware policy compiler expands into a sequence of typed stages. Each stage specifies which behaviors to run and how their

outputs should be combined. At runtime, the active stage runs three behaviors, semantic, predictive, and reactive, at different rates over a shared $SE(3)$ interface, which an action composer fuses into one command for compliant execution. The remainder of this section formalizes the setting and then details each level.

3.1 Problem Formulation

We consider long-horizon, contact-rich manipulation under partial observability. The robot is given a free-form language instruction $\mathcal{L}$ and must complete the described multi-stage task through closed-loop interaction. At each step t , the robot receives a multimodal observation $o_t = (I_t, \hat{\mathcal{X}}_t, \Psi_t)$, where I_t is RGB-D, $\hat{\mathcal{X}}_t$ is an object-centric *Scene Summary*, and Ψ_t contains tactile and force feedback. The *Scene Summary* includes object poses, grasp frames, calibrated task frames, point clouds, affordances, symbolic predicates, and robot state. The policy produces actions $a_t = ({}^W T_{\text{cmd}}(t), g_t)$, with target end-effector pose ${}^W T_{\text{cmd}}(t) \in SE(3)$ and gripper command g_t .

3.2 Stage Compilation: Policy Compiler and Interface

Given the instruction $\mathcal{L}$ and the initial Scene Summary $\hat{\mathcal{X}}_0$, a structured policy compiler converts the task into a sequence of stages. For example, the GPU-insertion task in Fig. 2 illustrates a multi-stage compilation, with four representative stages highlighted: pre-grasp, grasp, insert, and home. Each stage k is then specified by a *Stage Specification* $\mathcal{S}^k$ and a *Composition Specification* $\mathcal{C}^k$ using a fixed schema.

The *Stage Specification* contains the information needed to instantiate the active behaviors: the relational objective, reference frames, execution template, guard conditions, and recovery rules. We write this as $\mathcal{S}^k = (\Phi^k, F^k, E^k, G^k, R^k)$. The *Composition Specification* contains the information needed to fuse behavior outputs: the composition frame, axis ownership, residual bounds, and fallback behavior. We write this as $\mathcal{C}^k = (\mathcal{F}^k, \mathcal{O}^k, \mathcal{B}^k, \mathcal{H}^k)$. These fields parameterize the fixed behavior-composition rule defined in Sec. 3.4.

The specifications configure built-in behavior templates and controller profiles rather than introducing free-form code, continuous gains, or force thresholds. During execution, $\mathcal{S}^k$ parameterizes the online behaviors that produce the task-frame anchor, nominal motion, and tactile residual, while $\mathcal{C}^k$ parameterizes how these components are framed, projected, bounded, and safely composed. The controller compiler maps the selected profiles to numerical stiffness, admittance axes, residual bounds, and force/torque limits. Sec. 3.5 describes this deterministic controller compilation process, and Appendix 9 lists the calibrated profiles used in our experiments.

3.3 Per-Stage Parallel Action Generation

The three behaviors are specified by $\mathcal{S}^k$ and fused according to $\mathcal{C}^k$. Each emits only the component appropriate to its role (a frame, a motion, or a residual), rather than a complete command.

Semantic behavior: task-frame anchor. The semantic behavior emits a single anchor ${}^W T_{\mathcal{I}}^k$ from the world frame W to a stage-specific task frame $\mathcal{I}$, updated once per stage or on reobservation. It grounds Φ^k and F^k by minimizing a weighted $SE(3)$ alignment objective (Appendix 6) over LLM/VLM outputs, keypoints, calibrated fixtures, templates, or geometric constraints. The anchor is a task frame, not a trajectory, chosen so that motion and compliance decompose along its axes [51, 52]: this keeps axis ownership and stiffness diagonal and the tactile residual bounded. Because the anchor absorbs the object pose, the nominal motion and residual are relative and transfer across object poses and same-class instances ($SE(3)$ -equivariance) [53].

Predictive behavior: nominal motion. The predictive behavior emits $\mathcal{T}_{\text{traj}}^k(t)$, a continuous nominal motion in the task frame $\mathcal{I}$. A video world model [54], conditioned on I_t and the step instruction $\mathcal{L}_{\text{step}}$, samples K rollouts, and a VLM critic selects the rollout V^* that best satisfies the semantic and safety constraints. A 3D keypoint tracker [55] then lifts V^* to an object-centric reference relative to ${}^W T_{\mathcal{I}}^k$, forming $\mathcal{T}_{\text{traj}}^k(t)$. This is a motion prior, not a precise plan; any source that emits a task-frame trajectory can replace it.

Reactive behavior: tactile residual and force loop. The reactive behavior runs at 25 Hz. It emits a tactile residual for the composer and drives a force loop realized by the compliant controller. It also flags contact events (slip, excess force, or loss of progress) for the runtime supervisor. The *tactile residual* rejects in-hand slip: with NormalFlow [56], a GelSight Mini tracks the object’s pose relative to its nominal in-hand pose, and the residual corrects this sliding online [57], keeping the semantic and predictive targets valid relative to the object. The *force loop* regulates contact from the robot’s estimated end-effector wrench: the controller applies a virtual spring to maintain a target force for sustained-contact stages such as wiping, and position-based admittance to bound the contact force during insertion (Sec. 3.5). Appendix 7 details the tactile estimator and the compliant-control choices.

3.4 Multi-Rate $SE(3)$ Action Composer

At each controller tick the composer aligns components produced on different clocks: the anchor ${}^W T_{\mathcal{I}}^k$ is latched for the active stage, the nominal motion $\mathcal{I} T_{\text{traj}}^k(t)$ is interpolated, and the tactile residual $\Delta T_{\text{tac}}^k(t)$ uses the latest in-hand sliding estimate. It first anchors the nominal motion in the world frame,

$${}^W T_{\text{nom}}^k(t) = {}^W T_{\mathcal{I}}^k \mathcal{I} T_{\text{traj}}^k(t), \quad (1)$$

and then applies the tactile residual by right-multiplication,

$${}^W T_{\text{cmd}}^k(t) = {}^W T_{\text{nom}}^k(t) \Delta T_{\text{tac}}^k(t). \quad (2)$$

Here $\Delta T_{\text{tac}}^k(t)$ is the rigid-body transform that cancels the latest measured in-hand sliding (Appendix 7), expressed in the composition frame $\mathcal{F}^k$ from $\mathcal{C}^k$. The force loop is not composed here: it is realized by the compliant controller (Sec. 3.5), which regulates the end-effector wrench per stage. Because ${}^W T_{\text{cmd}}^k$ is recomputed each tick from the latest latched, sampled, and sensed inputs rather than integrated over time, drift is limited and each behavior updates at its own rate. If a component is missing or stale, the composer applies the per-stage fallback $\mathcal{H}^k$ from $\mathcal{C}^k$ (for example, holding the last valid anchor or zeroing the residual), so a dropped stream degrades gracefully rather than stalling the command.

3.5 Controller Compiler and Compliant Execution

The composer outputs a task-space command; the controller compiler turns $\mathcal{S}^k$ and $\mathcal{C}^k$ into the robot-level parameters that execute it: Cartesian stiffness and damping, impedance/admittance axis selection, force/torque and residual bounds, and guard and recovery rules, instantiated from the calibrated profile library keyed by $\mathcal{S}^k$ rather than emitted by the VLM (Appendices 8, 9). In free space, the controller tracks the anchored nominal motion with high stiffness and loose contact gates. During contact, each stage applies either Cartesian impedance (a virtual spring, to maintain a target force) or position-based admittance (to bound force), with the per-axis stiffness and force regime read from the controller profile in $\mathcal{S}^k$, the owned axes $\mathcal{O}^k$ from $\mathcal{C}^k$ (along which the tactile residual is admitted), and force limits enforced from the wrench estimate. Runtime guards advance the stage on success and trigger recovery, reobservation, or replanning on excess force, no progress, slip, or pose uncertainty.

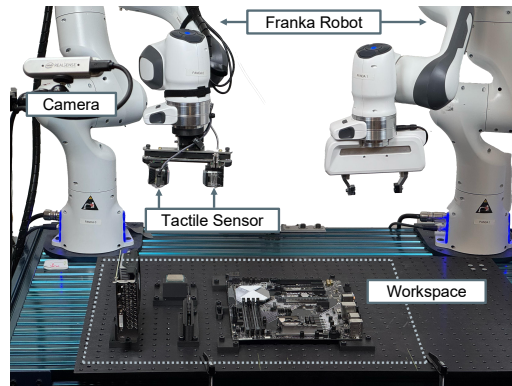


Figure 3: **Experimental Setup.** Two Franka Emika Panda robots; one manipulator carries a GelSight tactile sensor. The dashed outline indicates the workspace; a side-mounted camera provides visual perception.

Table 1: **Quantitative Success Rates (15 trials each).** (Left) Contact-rich assembly tasks against *VoxPoser* and $\pi_{0.5}$. (Right) Everyday manipulation tasks against $\pi_{0.5}$.

Assembly Task	VoxPoser	$\pi_{0.5}$	Ours	Everyday Task	$\pi_{0.5}$	Ours
Drill Insertion	0/15	0/15	15/15	Turn on the Lamp	0/15	5/15
RAM Insertion	0/15	0/15	14/15	Wipe the Whiteboard	0/15	8/15
CPU Placement	0/15	0/15	14/15	Cup $\rightarrow$ Plate	4/15	13/15
GPU Insertion	0/15	0/15	15/15	Clothes $\rightarrow$ Box	3/15	14/15

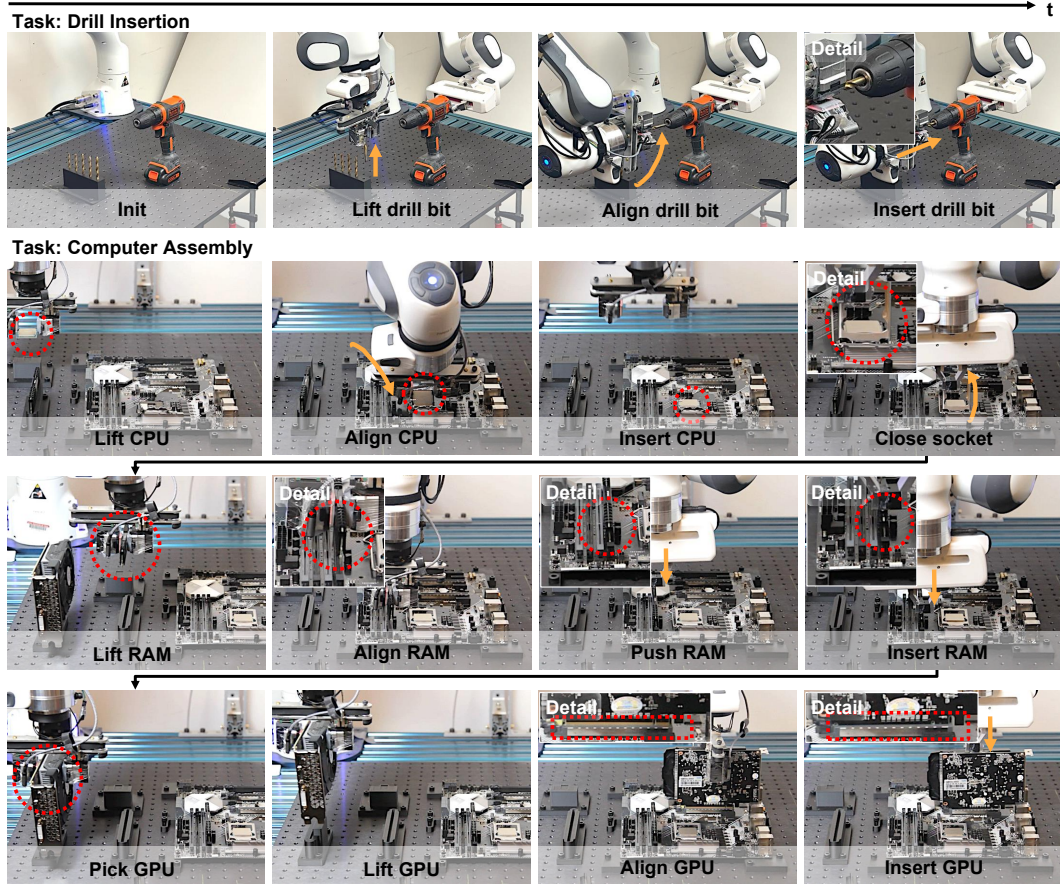


Figure 4: **Qualitative Rollouts of Contact-Rich Manipulation.** (Top) **Drill Insertion:** The system performs high-precision insertion with a 0.5 mm clearance. The reactive behavior enables sub-millimeter compliance and corrections in real-time, maintaining contact to prevent jamming. (Bottom) **Computer Assembly Sequence:** The system is able to sequentially execute CPU placement, RAM insertion, and GPU installation. By updating object-centric constraints, the same architecture transitions from delicate, force-sensitive CPU handling to the high-force requirements of RAM locking without task-specific retraining.

4 Experiments

To validate our framework, we conducted real-world experiments on long-horizon, high-precision robotic manipulation tasks. Our evaluation aims to address three core research questions: **(RQ1)** whether the asynchronous composition of the reactive behavior effectively rejects real-time perturbations to maintain sub-millimeter precision; **(RQ2)** whether the behavior composition architecture enables zero-shot transitions between tasks with diverse physical constraints without retraining; and **(RQ3)** whether a single set of behaviors transfers across the diverse contact regimes of CPU, RAM, GPU, and drill insertion without policy retraining or behavior-module redesign.

4.1 Experimental Setup

Hardware: We utilize a robotic setup with two Franka Emika Panda arms. One arm is equipped with a GelSight Mini tactile sensor for precision manipulation, while the other uses a standard

parallel-jaw gripper. External perception is provided by calibrated RealSense D415 cameras. Fig. 3 depicts the robots and sensor suite.

Tasks: We evaluate **CoStream** on two task groups. The first contains four high-precision assembly tasks inspired by desktop computer assembly: drill insertion with a 0.5 mm clearance, RAM insertion requiring high-force locking, CPU placement requiring gentle flat seating, and GPU insertion requiring long-edge alignment into a PCIe slot. These tasks evaluate tight-clearance contact, robustness, and sequential assembly. The second group contains four everyday manipulation tasks: turning on a lamp, wiping a whiteboard, moving a cup to a plate, and placing clothes into a box, which evaluate broader manipulation behaviors beyond precision assembly.

Baselines We compare against two representative external baselines that instantiate dominant alternatives to **CoStream**: VoxPoser [23], a modular vision-language planning pipeline, and $\pi_{0.5}$ [12], a monolithic VLA policy used without task-specific training. These released foundation-model baselines with comparable tactile or force-feedback inputs are not available; we therefore evaluate these accessible methods under their native visual/language interfaces. These comparisons test whether current modular and monolithic foundation-model paradigms can solve the same long-horizon, contact-rich tasks under their standard assumptions. We use internal ablations to isolate the contribution of individual **CoStream** behaviors and sensing streams.

4.2 Results and Analysis

Quantitative success rates. Table 1 reports real-world success across high-precision assembly and everyday manipulation tasks. In tight-clearance assembly, **CoStream** succeeds consistently, while VoxPoser and $\pi_{0.5}$ fail to complete the tasks. On everyday tasks, **CoStream** improves over $\pi_{0.5}$ across all settings, with larger gains on object transfer and smaller gains on lamp switching and whiteboard wiping. Since each setting uses 15 real-world trials, the numbers should be interpreted as task-level evidence rather than precise estimates of success probability. The large gap on assembly tasks nevertheless indicates a consistent qualitative difference between open-loop visual planning, monolithic VLA control, and closed-loop behavior composition.

Failure analysis of baselines. VoxPoser fails before contact. Its affordance maps are constructed from a single visual snapshot and never updated; once the planner commits a trajectory, deviations from the expected geometry have no path back into the system. In our tasks the maps miss the slot orientation by several degrees, and the open-loop trajectory drives the grasped part into the rim of the hole. $\pi_{0.5}$ fails during contact: mapping observations directly to commands, it cannot separate a contact event from perceptual ambiguity, and on contact responds with high-frequency motion that destabilizes alignment. This is exactly the failure mode **CoStream**’s reactive behavior eliminates: contact corrections live in a dedicated behavior with their own sensing, rate, and composition term.

Failure analysis of **CoStream.** The single CPU failure occurred at seating, when the chip rotated outside the tactile contact patch before the reactive term registered it; the single RAM failure occurred at locking, when the downward force briefly exceeded the compiled force-limit envelope. Both sit at the boundary of the reactive behavior’s sensing or compliance envelope, not at the composition, though $N = 15$ is small.

Qualitative rollouts (RQ1 & RQ2). Fig. 4 shows two rollouts. In drill insertion, the bit stays centered in the 0.5 mm clearance by compliant adaptation while baselines jam. The second is a continuous CPU–RAM–GPU assembly without retraining, where only the object-centric constraints change between subtasks while the behaviors and control loops stay fixed from delicate CPU placement to high-force RAM locking.

Table 2: **Ablation Study on Drill Insertion.** Removing the reactive behavior yields a significantly lower success rate, showing that contact feedback is essential for sub-millimeter precision.

Method	Success Rate
Ours (Full Framework)	15/15
Ours w/o Reactive Behavior	3/15

Robustness to human perturbation (RQ1). We rotate the held object inside the gripper during insertion to simulate slip. The reactive behavior detects the deviation from the reference normal

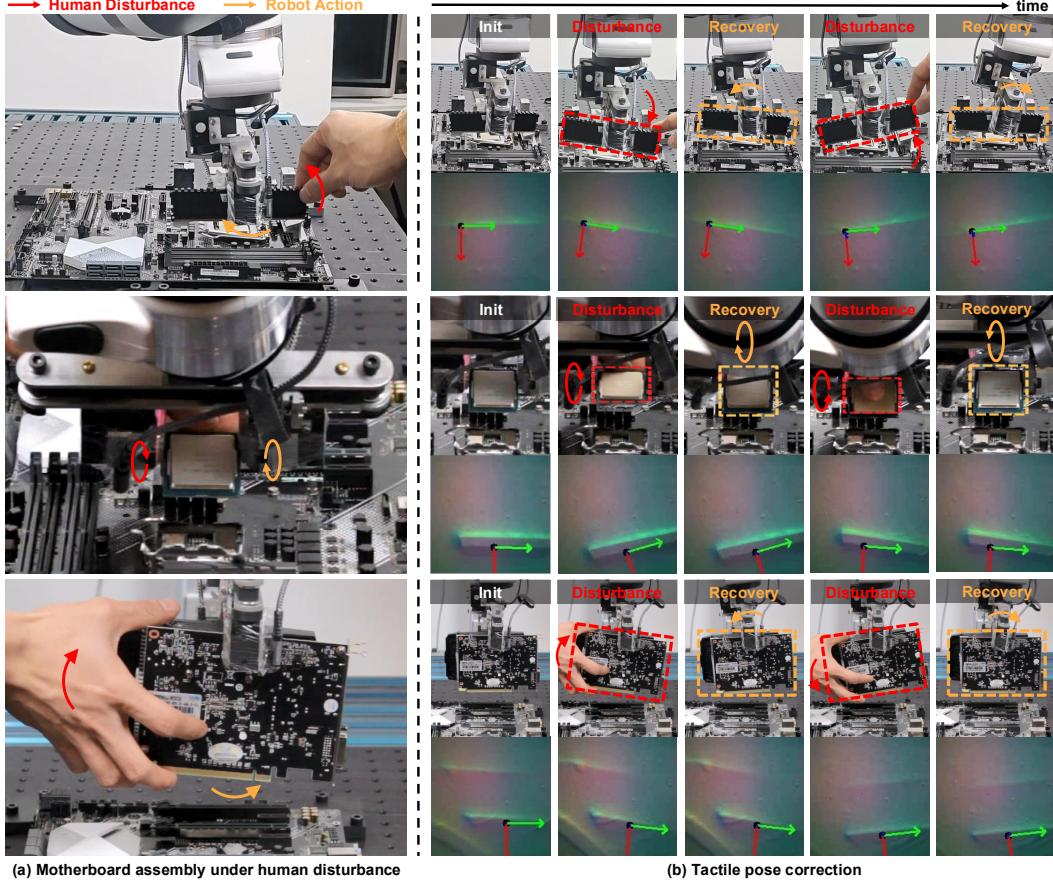


Figure 5: **Robustness to Human Perturbation via Tactile Feedback.** (Left) Snapshots of the robot recovering from manual object displacement while inserting a RAM module (top), CPU (middle), and GPU (bottom). (Right) Per-component reactive alignment: each first row shows the human-induced in-gripper rotation, and each second row the raw tactile readings and derived real-time pose estimate used to realign.

map and reorients the end-effector to realign (Fig. 5, left, across RAM, CPU, and GPU). The right panel breaks down recovery across the three components (thin RAM surfaces, a small CPU footprint, the GPU’s mass). **Mechanism ablation (RQ1).** Without the reactive behavior on *Drill Insertion* (no tactile or force feedback), success collapses (Table 2): the controller cannot reject the angular misalignments inside the 0.5 mm clearance, succeeding only under near-perfect initial alignment.

5 Limitations and Conclusion

Limitations. *CoStream* inherits its perception stack: pose estimates depend on upstream modules, and visual prediction degrades when imagined rollouts drift from the scene. Our experiments target structured rigid-object manipulation with calibrated perception, reusable templates, and explicit stage specifications; deformable and articulated objects, dexterous in-hand manipulation, and ambiguous goals remain out of scope. The reactive behavior is limited by tactile coverage and end-effector wrench estimation, so severe slip, out-of-patch contact, or inaccurate force estimates can still cause failure. Extending the composition interface to less structured settings and broader object categories is important future work.

Conclusion. *CoStream* decomposes long-horizon contact-rich manipulation into a semantic, a predictive, and a reactive behavior, composed by right-multiplication into one pose command per control step and run by a compliant controller. The same behaviors clear four assembly tasks, recover from manual perturbation, and transfer across components without retraining, a regime that monolithic policies and modular pipelines do not reach.

Acknowledgments

We thank Jiayuan Mao and Pengfei Ye for helpful discussions. This work has been made possible in part by a gift from the Chan Zuckerberg Initiative Foundation to establish the Kempner Institute for the Study of Natural and Artificial Intelligence at Harvard University. This work is financially supported by Toyota Research Institute and Amazon Science Hub.

References

- [1] H. Shi, H. Xu, Z. Huang, Y. Li, and J. Wu. Robocraft: Learning to see, simulate, and shape elasto-plastic objects with graph networks, 2022.
- [2] H. Shi, H. Xu, S. Clarke, Y. Li, and J. Wu. Robocook: Long-horizon elasto-plastic object manipulation with diverse tools. In *7th Annual Conference on Robot Learning*, 2023.
- [3] H. Chen, Y. Niu, K. Hong, S. Liu, Y. Wang, Y. Li, and K. R. Driggs-Campbell. Predicting object interactions with behavior primitives: An application in stowing tasks. In *7th Annual Conference on Robot Learning*, 2023.
- [4] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choromanski, T. Ding, D. Driess, A. Dubey, C. Finn, P. Florence, C. Fu, M. G. Arenas, K. Gopalakrishnan, K. Han, K. Hausman, A. Herzog, J. Hsu, B. Ichter, A. Irpan, N. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, I. Leal, L. Lee, T.-W. E. Lee, S. Levine, Y. Lu, H. Michalewski, I. Mordatch, K. Pertsch, K. Rao, K. Reymann, M. Ryoo, G. Salazar, P. Sanketi, P. Sermanet, J. Singh, A. Singh, R. Soricut, H. Tran, V. Vanhoucke, Q. Vuong, A. Wahid, S. Welker, P. Wohlhart, J. Wu, F. Xia, T. Xiao, P. Xu, S. Xu, T. Yu, and B. Zitkovich. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In *arXiv preprint arXiv:2307.15818*, 2023.
- [5] Open X-Embodiment Collaboration, A. Padalkar, A. Pooley, A. Jain, A. Bewley, A. Herzog, A. Irpan, A. Khazatsky, A. Rai, A. Singh, A. Brohan, A. Raffin, A. Wahid, B. Burgess-Limerick, B. Kim, B. Schölkopf, B. Ichter, C. Lu, C. Xu, C. Finn, C. Xu, C. Chi, C. Huang, C. Chan, C. Pan, C. Fu, C. Devin, D. Driess, D. Pathak, D. Shah, D. Büchler, D. Kalashnikov, D. Sadigh, E. Johns, F. Ceola, F. Xia, F. Stulp, G. Zhou, G. S. Sukhatme, G. Salhotra, G. Yan, G. Schiavi, H. Su, H.-S. Fang, H. Shi, H. B. Amor, H. I. Christensen, H. Furuta, H. Walke, H. Fang, I. Mordatch, I. Radosavovic, I. Leal, J. Liang, J. Kim, J. Schneider, J. Hsu, J. Bohg, J. Bingham, J. Wu, J. Wu, J. Luo, J. Gu, J. Tan, J. Oh, J. Malik, J. Tompson, J. Yang, J. J. Lim, J. Silvério, J. Han, K. Rao, K. Pertsch, K. Hausman, K. Go, K. Gopalakrishnan, K. Goldberg, K. Byrne, K. Oslund, K. Kawaharazuka, K. Zhang, K. Majd, K. Rana, K. Srinivasan, L. Y. Chen, L. Pinto, L. Tan, L. Ott, L. Lee, M. Tomizuka, M. Du, M. Ahn, M. Zhang, M. Ding, M. K. Srirama, M. Sharma, M. J. Kim, N. Kanazawa, N. Hansen, N. Heess, N. J. Joshi, N. Suenderhauf, N. D. Palo, N. M. M. Shafiullah, O. Mees, O. Kroemer, P. R. Sanketi, P. Wohlhart, P. Xu, P. Sermanet, P. Sundaresan, Q. Vuong, R. Rafailov, R. Tian, R. Doshi, R. Martín-Martín, R. Mendonca, R. Shah, R. Hoque, R. Julian, S. Bustamante, S. Kirmani, S. Levine, S. Moore, S. Bahl, S. Dass, S. Song, S. Xu, S. Haldar, S. Adebola, S. Guist, S. Nasiriany, S. Schaal, S. Welker, S. Tian, S. Dasari, S. Belkhale, T. Osa, T. Harada, T. Matsushima, T. Xiao, T. Yu, T. Ding, T. Davchev, T. Z. Zhao, T. Armstrong, T. Darrell, V. Jain, V. Vanhoucke, W. Zhan, W. Zhou, W. Burgard, X. Chen, X. Wang, X. Zhu, X. Li, Y. Lu, Y. Chebotar, Y. Zhou, Y. Zhu, Y. Xu, Y. Wang, Y. Bisk, Y. Cho, Y. Lee, Y. Cui, Y. hua Wu, Y. Tang, Y. Zhu, Y. Li, Y. Iwasawa, Y. Matsuo, Z. Xu, and Z. J. Cui. Open X-Embodiment: Robotic learning datasets and RT-X models. <https://arxiv.org/abs/2310.08864>, 2023.
- [6] J. Huang, S. Yong, X. Ma, X. Linghu, P. Li, Y. Wang, Q. Li, S.-C. Zhu, B. Jia, and S. Huang. An embodied generalist agent in 3d world. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2024.
- [7] D. A. Pomerleau. ALVINN: An autonomous land vehicle in a neural network. *Advances in neural information processing systems*, 1, 1988.

- [8] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Proceedings of Robotics: Science and Systems (RSS)*, 2023.
- [9] P. Florence, C. Lynch, A. Zeng, O. A. Ramirez, A. Wahid, L. Downs, A. Wong, J. Lee, I. Mordatch, and J. Tompson. Implicit behavioral cloning. In *5th Annual Conference on Robot Learning*, 2021.
- [10] Octo Model Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, C. Xu, J. Luo, T. Kreiman, Y. Tan, L. Y. Chen, P. Sanketi, Q. Vuong, T. Xiao, D. Sadigh, C. Finn, and S. Levine. Octo: An open-source generalist robot policy. In *Proceedings of Robotics: Science and Systems*, Delft, Netherlands, 2024.
- [11] K. Black, N. Brown, D. Driess, A. Esmail, M. R. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, L. X. Shi, L. Smith, J. Tanner, Q. Vuong, A. Walling, H. Wang, and U. Zhilinsky. π_0 : A Vision-Language-Action Flow Model for General Robot Control. In *Proceedings of Robotics: Science and Systems*, LosAngeles, CA, USA, June 2025. doi: [10.15607/RSS.2025.XXI.010](https://doi.org/10.15607/RSS.2025.XXI.010).
- [12] K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. R. Equi, C. Finn, N. Fusai, M. Y. Galliker, D. Ghosh, L. Groom, K. Hausman, b. ichter, S. Jakubczak, T. Jones, L. Ke, D. LeBlanc, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, A. Z. Ren, L. X. Shi, L. Smith, J. T. Springenberg, K. Stachowicz, J. Tanner, Q. Vuong, H. Walke, A. Walling, H. Wang, L. Yu, and U. Zhilinsky. $\pi_{0.5}$: a vision-language-action model with open-world generalization. In J. Lim, S. Song, and H.-W. Park, editors, *Proceedings of The 9th Conference on Robot Learning*, volume 305 of *Proceedings of Machine Learning Research*, pages 17–40. PMLR, 27–30 Sep 2025. URL <https://proceedings.mlr.press/v305/black25a.html>.
- [13] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. P. Foster, P. R. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn. Openvla: An open-source vision-language-action model. In P. Agrawal, O. Kroemer, and W. Burgard, editors, *Proceedings of The 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pages 2679–2713. PMLR, 06–09 Nov 2025. URL <https://proceedings.mlr.press/v270/kim25c.html>.
- [14] M. J. Kim, C. Finn, and P. Liang. Fine-Tuning Vision-Language-Action Models: Optimizing Speed and Success. In *Proceedings of Robotics: Science and Systems*, LosAngeles, CA, USA, June 2025. doi:[10.15607/RSS.2025.XXI.017](https://doi.org/10.15607/RSS.2025.XXI.017).
- [15] J. Bjorck, F. Castañeda, N. Cherniadev, X. Da, R. Ding, L. J. Fan, Y. Fang, D. Fox, F. Hu, S. Huang, J. Jang, Z. Jiang, J. Kautz, K. Kundalia, L. Lao, Z. Li, Z. Lin, K. Lin, G. Liu, E. Llontop, L. Magne, A. Mandlekar, A. Narayan, S. Nasiriany, S. Reed, Y. L. Tan, G. Wang, Z. Wang, J. Wang, Q. Wang, J. Xiang, Y. Xie, Y. Xu, Z. Xu, S. Ye, Z. Yu, A. Zhang, H. Zhang, Y. Zhao, R. Zheng, and Y. Zhu. Gr00t n1: An open foundation model for generalist humanoid robots, 2025. URL <https://arxiv.org/abs/2503.14734>.
- [16] H. A. Simon. *The Sciences of the Artificial*. MIT Press, Cambridge, MA, 3 edition, Sept. 1996. ISBN 9780262264495.
- [17] R. Brooks. A robust layered control system for a mobile robot. *IEEE Journal on Robotics and Automation*, 2(1):14–23, 1986. doi:[10.1109/JRA.1986.1087032](https://doi.org/10.1109/JRA.1986.1087032).
- [18] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, C. Fu, K. Gopalakrishnan, K. Hausman, A. Herzog, D. Ho, J. Hsu, J. Ibarz, B. Ichter, A. Irpan, E. Jang, R. J. Ruano, K. Jeffrey, S. Jesmonth, N. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, K.-H. Lee, S. Levine, Y. Lu, L. Luu, C. Parada, P. Pastor, J. Quiambao, K. Rao, J. Rettinghouse, D. Reyes,

- P. Sermanet, N. Sievers, C. Tan, A. Toshev, V. Vanhoucke, F. Xia, T. Xiao, P. Xu, S. Xu, M. Yan, and A. Zeng. Do as i can and not as i say: Grounding language in robotic affordances. In *arXiv preprint arXiv:2204.01691*, 2022.
- [19] Y. Hu, F. Lin, T. Zhang, L. Yi, and Y. Gao. Look before you leap: Unveiling the power of gpt-4v in robotic vision-language planning. *arXiv preprint arXiv:2311.17842*, 2023.
 - [20] Y. Hong, H. Zhen, P. Chen, S. Zheng, Y. Du, Z. Chen, and C. Gan. 3d-llm: Injecting the 3d world into large language models. In *Advances in Neural Information Processing Systems*, volume 36, pages 20482–20494, 2023.
 - [21] J. Duan, W. Yuan, W. Pumacay, Y. R. Wang, K. Ehsani, D. Fox, and R. Krishna. Manipulate-anything: Automating real-world robots using vision-language models. *arXiv preprint arXiv:2406.18915*, 2024.
 - [22] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. R. Florence, and A. Zeng. Code as policies: Language model programs for embodied control. *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 9493–9500, 2022.
 - [23] W. Huang, C. Wang, R. Zhang, Y. Li, J. Wu, and L. Fei-Fei. Voxposer: Composable 3d value maps for robotic manipulation with language models. In *Conference on Robot Learning (CoRL)*, 2023.
 - [24] W. Huang, C. Wang, Y. Li, R. Zhang, and L. Fei-Fei. Rekep: Spatio-temporal reasoning of relational keypoint constraints for robotic manipulation. In *8th Annual Conference on Robot Learning*, 2024.
 - [25] K. Fang, F. Liu, P. Abbeel, and S. Levine. Moka: Open-world robotic manipulation through mark-based visual prompting. *Robotics: Science and Systems (RSS)*, 2024.
 - [26] W. Yuan, J. Duan, V. Blukis, W. Pumacay, R. Krishna, A. Murali, A. Mousavian, and D. Fox. Robopoint: A vision-language model for spatial affordance prediction for robotics. *arXiv preprint arXiv:2406.10721*, 2024.
 - [27] S. Nasiriany, F. Xia, W. Yu, T. Xiao, J. Liang, I. Dasgupta, A. Xie, D. Driess, A. Wahid, Z. Xu, et al. Pivot: Iterative visual prompting elicits actionable knowledge for vlms. *arXiv preprint arXiv:2402.07872*, 2024.
 - [28] H. Liu, S. Yao, H. Chen, J. Gao, J. Mao, J.-B. Huang, and Y. Du. Simpack: Simulation-enabled action planning using vision-language models, 2025.
 - [29] Y. She, S. Wang, S. Dong, N. Sunil, A. Rodriguez, and E. Adelson. Cable manipulation with a tactile-reactive gripper. *The International Journal of Robotics Research*, 40(12-14):1385–1401, 2021. doi:10.1177/02783649211027233.
 - [30] Y. Yuan, H. Che, Y. Qin, B. Huang, Z.-H. Yin, K.-W. Lee, Y. Wu, S.-C. Lim, and X. Wang. Robot synesthesia: In-hand manipulation with visuotactile sensing. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6558–6565. IEEE, 2024.
 - [31] I. Guzey, Y. Dai, B. Evans, S. Chintala, and L. Pinto. See to touch: Learning tactile dexterity through visual incentives. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 13825–13832. IEEE, 2024.
 - [32] B. Ai, S. Tian, H. Shi, Y. Wang, C. Tan, Y. Li, and J. Wu. Robopack: Learning tactile-informed dynamics models for dense packing. 2024. URL <https://arxiv.org/abs/2407.01418>.
 - [33] M. Oller, D. Berenson, and N. Fazeli. Tactile-driven non-prehensile object manipulation via extrinsic contact mode control. *arXiv preprint arXiv:2405.18214*, 2024.

- [34] R. Ye, Y. Hu, Y. A. Bian, L. Kulm, and T. Bhattacharjee. Morpheus: a multimodal one-armed robot-assisted peeling system with human users in-the-loop. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 9540–9547. IEEE, 2024.
- [35] W. Hu, B. Huang, W. W. Lee, S. Yang, Y. Zheng, and Z. Li. Dexterous in-hand manipulation of slender cylindrical objects through deep reinforcement learning with tactile sensing, 2023.
- [36] Z. Yu, W. Xu, S. Yao, J. Ren, T. Tang, Y. Li, G. Gu, and C. Lu. Precise robotic needle-threading with tactile perception and reinforcement learning. In *Conference on Robot Learning*, pages 3266–3276. PMLR, 2023.
- [37] T. Lin, Y. Zhang, Q. Li, H. Qi, B. Yi, S. Levine, and J. Malik. Learning visuotactile skills with two multifingered hands. *arXiv:2404.16823*, 2024.
- [38] B. Huang, Y. Wang, X. Yang, Y. Luo, and Y. Li. 3d vitac: learning fine-grained manipulation with visuo-tactile sensing. In *Proceedings of Robotics: Conference on Robot Learning (CoRL)*, 2024.
- [39] W. v. d. Bogert, M. Iyengar, and N. Fazeli. Built different: Tactile perception to overcome cross-embodiment capability differences in collaborative manipulation. *arXiv preprint arXiv:2409.14896*, 2024.
- [40] K. Yu, Y. Han, Q. Wang, V. Saxena, D. Xu, and Y. Zhao. Mimictouch: Leveraging multi-modal human tactile demonstrations for contact-rich manipulation. In *8th Annual Conference on Robot Learning*, 2024.
- [41] H. Chen, J. Xu, H. Chen, K. Hong, B. Huang, C. Liu, J. Mao, Y. Li, Y. Du, and K. R. Driggs-Campbell. Multi-modal manipulation via multi-modal policy consensus. *ArXiv*, abs/2509.23468, 2025.
- [42] S. Tian, F. Ebert, D. Jayaraman, M. Mudigonda, C. Finn, R. Calandra, and S. Levine. Manipulation by feel: Touch-based control with deep predictive models. *arXiv preprint arXiv:1903.04128*, 2019.
- [43] C. R. Garrett, R. Chitnis, R. Holladay, B. Kim, T. Silver, L. P. Kaelbling, and T. Lozano-Pérez. Integrated task and motion planning. *Annual Review of Control, Robotics, and Autonomous Systems*, 4(Volume 4, 2021):265–293, 2021. ISSN 2573-5144.
- [44] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu, et al. RT-1: Robotics transformer for real-world control at scale. In *Proceedings of Robotics: Science and Systems (RSS)*, 2023.
- [45] H. Chen, J. Xu, L. Sheng, T. Ji, S. Liu, Y. Li, and K. Driggs-Campbell. Learning coordinated bimanual manipulation policies using state diffusion and inverse dynamics models. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, 2025.
- [46] H. Zhen, X. Qiu, P. Chen, J. Yang, X. Yan, Y. Du, Y. Hong, and C. Gan. 3d-vla: 3d vision-language-action generative world model. *arXiv preprint arXiv:2403.09631*, 2024.
- [47] L. Wang, J. Zhao, Y. Du, E. H. Adelson, and R. Tedrake. Poco: Policy composition from and for heterogeneous robot learning, 2024.
- [48] C. Liu, H. Chen, S. H. Høeg, S. Yao, Y. Li, K. Hauser, and Y. Du. Flexible multitask learning with factorized diffusion policy, 2025.
- [49] A. Ajay, S. Han, Y. Du, S. Li, A. Gupta, T. Jaakkola, J. Tenenbaum, L. Kaelbling, A. Srivastava, and P. Agrawal. Compositional foundation models for hierarchical planning. *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

- [50] Y. Du, M. Yang, P. Florence, F. Xia, A. Wahid, B. Ichter, P. Sermanet, T. Yu, P. Abbeel, J. B. Tenenbaum, et al. Video language planning. *International Conference on Learning Representations*, 2024.
- [51] M. T. Mason. Compliance and force control for computer controlled manipulators. *IEEE Transactions on Systems, Man, and Cybernetics*, 11(6):418–432, 1981.
- [52] H. Bruyninckx and J. De Schutter. Specification of force-controlled actions in the “task frame formalism”—a synthesis. *IEEE Transactions on Robotics and Automation*, 12(4):581–589, 1996.
- [53] A. Simeonov, Y. Du, A. Tagliasacchi, J. B. Tenenbaum, A. Rodriguez, P. Agrawal, and V. Sitzmann. Neural descriptor fields: SE(3)-equivariant object representations for manipulation. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2022.
- [54] B. Chen, T. Zhang, H. Geng, K. Song, C. Zhang, P. Li, W. T. Freeman, J. Malik, P. Abbeel, R. Tedrake, V. Sitzmann, and Y. Du. Large video planner enables generalizable robot control, 2025. URL <https://arxiv.org/abs/2512.15840>.
- [55] Y. Xiao, J. Wang, N. Xue, N. Karaev, I. Makarov, B. Kang, X. Zhu, H. Bao, Y. Shen, and X. Zhou. Spatialtrackerv2: 3d point tracking made easy. In *ICCV*, 2025.
- [56] H.-J. Huang, M. Kaess, and W. Yuan. Normalflow: Fast, robust, and accurate contact-based object 6dof pose tracking with vision-based tactile sensors. *IEEE Robotics and Automation Letters*, 10(1):452–459, Jan. 2025. ISSN 2377-3774. doi:10.1109/lra.2024.3505815. URL <http://dx.doi.org/10.1109/LRA.2024.3505815>.
- [57] W.-H. Chen, J. Yang, L. Guo, and S. Li. Disturbance-observer-based control and related methods – an overview. *IEEE Transactions on Industrial Electronics*, 63(2):1083–1095, 2016.
- [58] S. Baker and I. Matthews. Lucas-kanade 20 years on: A unifying framework. *International Journal of Computer Vision*, 56(3):221–255, feb 2004. ISSN 1573-1405. doi:10.1023/B:VISI.0000011205.11775.fd.
- [59] W. Shen, N. Kumar, S. Chintalapudi, J. Wang, C. Watson, E. Hu, J. Cao, D. Jayaraman, L. P. Kaelbling, and T. Lozano-Pérez. Tiptop: A modular open-vocabulary planning system for robotic manipulation, 2026.
- [60] B. Wen, M. Trepte, J. Aribido, J. Kautz, O. Gallo, and S. Birchfield. Foundationstereo: Zero-shot stereo matching, 2025.
- [61] W. Yuan, A. Murali, A. Mousavian, and D. Fox. M2t2: Multi-task masked transformer for object-centric pick and place. In *Conference on Robot Learning (CoRL)*, 2023.
- [62] G. R. Team, S. Abeyruwan, J. Ainslie, J.-B. Alayrac, M. G. Arenas, T. Armstrong, A. Balakrishna, R. Baruch, M. Bauza, M. Blokzijl, et al. Gemini robotics: Bringing AI into the physical world. *arXiv preprint arXiv:2503.20020*, 2025.
- [63] Google DeepMind. Gemini robotics-er 1.6: Powering real-world robotics tasks through enhanced embodied reasoning. <https://deepmind.google/blog/gemini-robotics-er-1-6/>, 2026. Accessed: 2026-05-29.
- [64] N. Carion, L. Gustafson, Y.-T. Hu, S. Debnath, R. Hu, D. Suris, C. Ryali, K. V. Alwala, H. Khedr, A. Huang, et al. SAM 3: Segment anything with concepts. *arXiv preprint arXiv:2511.16719*, 2025.

6 Semantic behavior: grounding solver

The semantic behavior grounds the task-frame anchor ${}^W T_{\mathcal{I}}^k$ by minimizing a weighted sum of M differentiable geometric residuals over $SE(3)$,

$${}^W T_{\mathcal{I}}^{k*} = \arg \min_{{}^W T_{\mathcal{I}} \in SE(3)} \sum_{j=1}^M w_j \mathcal{J}_j({}^W T_{\mathcal{I}}; \hat{\mathcal{X}}_t).$$

Here $\hat{\mathcal{X}}_t$ is the Scene Summary, which supplies the object and fixture poses and detected keypoints used by the residuals. The active residuals $\{\mathcal{J}_j\}$ and weights $\{w_j\}$ are instantiated by the compiler from the stage objective Φ^k and grounding sources F^k ; each $\mathcal{J}_j$ encodes one geometric relation the anchor must satisfy, such as point alignment, axis or orientation alignment, or a standoff offset. A typical point-alignment residual ties a keypoint expressed in the task frame to one on the target object,

$$\mathcal{J}_{\text{align}}({}^W T_{\mathcal{I}}) = \left\| {}^W T_{\mathcal{I}} \tilde{\mathbf{p}}_{\mathcal{I}} - {}^W T_{\text{obj}} \tilde{\mathbf{p}}_{\text{obj}} \right\|_2^2,$$

where ${}^W T_{\text{obj}}$ is the target object (or fixture) pose from $\hat{\mathcal{X}}_t$, and $\mathbf{p}_{\mathcal{I}}, \mathbf{p}_{\text{obj}} \in \mathbb{R}^3$ are corresponding keypoints expressed in the task frame $\mathcal{I}$ and the object frame, respectively, with $\tilde{\mathbf{p}}$ denoting homogeneous coordinates. We initialize with Basin Hopping, refine with Sequential Least Squares Programming (SLSQP), and warm-start across timesteps for temporal continuity.

7 Reactive behavior: tactile residual and force loop

Tactile residual (in-hand slip rejection). We directly track the object’s pose relative to its nominal in-hand pose with NormalFlow [56], a per-frame inverse-compositional estimator [58]. Each frame solves for the warp $\Delta\xi$ that aligns the current tactile normal map to the nominal-pose reference,

$$\Delta\xi^* = \arg \min_{\Delta\xi} \sum_{\mathbf{u} \in \Omega} \|R(\Delta\xi)^{-1} I_{\text{obs}}(W(\mathbf{u}; \xi)) - I_{\text{ref}}(\mathbf{u})\|^2,$$

where Ω is the contact region and W is the warp induced by twist ξ . Because surface normal maps are scale-ambiguous in depth, we decouple the z component and recover it by averaging the depth discrepancy over the contact patch,

$$\Delta z = \frac{1}{|\Omega|} \sum_{\mathbf{u} \in \Omega} [D_{\text{obs}}(W(\mathbf{u}; \xi^*)) - (\mathbf{R}_{\xi^*} \cdot \mathbf{q}(\mathbf{u}) + \mathbf{t}_{\xi^*})_z],$$

where D_{obs} is the depth map from Poisson integration and $\mathbf{q}(\mathbf{u})$ is the unwarped 3D surface point at pixel $\mathbf{u}$. The recovered transform $(\Delta\xi^*, \Delta z)$ is the object’s pose relative to its nominal in-hand pose, i.e., its real-time in-hand sliding. The tactile residual is the end-effector motion that compensates this sliding online [57]; it is the rigid-body transform ΔT_{tac}^k applied in Eq. (2).

Force loop (compliant contact regulation). The force loop is realized by the compliant controller from the robot’s estimated end-effector wrench, recovered from the Franka link-side joint-torque sensors mapped through the manipulator Jacobian ($F_{\text{ext}} = (J^\top)^+ \tau_{\text{ext}}$), with no wrist force/torque sensor. For stages that must sustain a contact force, such as wiping or pressing, a virtual-spring (Cartesian impedance) law regulates the measured wrench toward a target, giving approximate force regulation without dedicated force sensing. For insertion, where the goal is instead to bound contact force, position-based admittance maps the measured wrench to a bounded pose correction. We use position-based admittance rather than torque-level impedance because the end-effector carries additional cameras, tactile sensors, mounts, and cables: the resulting payload and friction mismatch degrade torque-level Cartesian tracking, whereas admittance preserves the robot’s inner position servo while adding bounded, force-responsive motion that lowers the risk of jamming or damaging parts.

8 Controller compilation

The controller compiler converts $\mathcal{S}^k$ and $\mathcal{C}^k$ into a control packet for the low-level compliant controller,

$$U^k(t) = ({}^W T_{\text{cmd}}^k(t), \xi_{\text{cmd}}^k(t), K^k, D^k, A^k, \mathcal{B}^k, G^k, R^k),$$

where $\xi_{\text{cmd}}^k(t)$ is the commanded task-space twist, K^k and D^k are Cartesian stiffness and damping, A^k selects impedance/admittance behavior along controlled axes, $\mathcal{B}^k$ contains force, torque, residual-translation, residual-rotation, and timeout bounds, and G^k, R^k define guard evaluation and fallback behavior. Stiffness is high in free space and lowered, or switched to admittance, on the axes the Composition Specification marks compliant.

9 Controller and guard profiles

The policy compiler chooses a per-stage controller profile and guard profile by identifier. The controller compiler then maps each symbolic level (*low*, *medium*, *high*) to a robot-specific numerical value, calibrated once per platform from the robot, end-effector, and object-class limits with a safety margin. The LLM/VLM never produces these numbers directly: it picks a profile, not a gain or a threshold. This keeps the action space auditable and the contact behavior within calibrated safety limits. Tables 3 and 4 give a representative library; the assembly stages in this paper use the approach, insertion, press, and wiping profiles.

Table 3: **Controller-profile library.** Each profile fixes the control mode and a symbolic stiffness/compliance pattern; the numerical stiffness, damping, and admittance gains are calibrated per platform.

Profile	Mode	Compliant axes	Stiffness / force regime
free_space	Impedance	none	High translation and rotation
guarded_approach	Impedance	progress	Medium, gated on contact
compliant_insertion	Impedance / admittance	lateral, roll, pitch	Low lateral and roll/pitch, medium progress
high_force_press	Admittance	normal	Low normal stiffness, force-controlled normal axis
surface_wiping	Admittance	surface normal	Force-controlled normal, free tangential motion

Table 4: **Guard-profile library.** Each profile fixes symbolic force/torque limits and a no-progress timeout; numerical thresholds are calibrated with a safety margin.

Profile	Normal-force limit	Torque limit	No-progress timeout
delicate_placement	Low	Low	Short
rigid_insertion	Medium	Medium-low	Medium
high_force_locking	High (bounded)	Medium	Short
surface_wiping	Medium band	Medium	Disabled (sustained contact)

10 Perception module

The Scene Summary $\hat{\mathcal{X}}$ consumed by the policy compiler (Sec. 3.2) is produced by an open-vocabulary perception stack. We adopt the two-branch design of TiPToP [59]: a 3D vision branch recovers metric geometry, and a semantic branch grounds the instruction to objects. We then upgrade both branches and extend their fused output, as detailed below.

3D vision branch. From the calibrated RGB-D streams, FoundationStereo [60] estimates a dense scene point cloud, and M2T2 [61] predicts candidate 6-DoF grasps from it. Masking the dense cloud with the semantic branch below yields per-object point clouds, and convex-hull reconstruction completes object geometry, following TiPToP [59]. The resulting meshes support collision checking and grasp-frame estimation.

Semantic branch. Gemini Robotics-ER 1.6 [62, 63] detects task-relevant objects and grounds the instruction $\mathcal{L}$ to symbolic goal propositions. SAM 3 [64] then lifts these detections to per-object masks through concept prompts that index into the point cloud. This branch supplies the object identities, language-grounded targets, and symbolic predicates of $\hat{\mathcal{X}}$.

Scene Summary construction. Fusing the two branches yields an object-centric record per task-relevant object: pose, completed mesh, and candidate grasps. We annotate each record with three further fields. *Affordances*, such as an insertable bore, a slot, or a graspable edge, are attached to specific object parts. *Interaction and task frames* are attached to those affordances, and seed the semantic behavior’s anchor ${}^W T_{\mathcal{I}}^k$. *Available geometry sources*, such as calibrated fixtures, object-centric templates, or estimated poses, are tagged per object. These tags let the compiler judge which frames are reliable for a high-precision stage. Together, the three fields let $\hat{\mathcal{X}}$ compile into Stage and Composition Specifications without a separate grounding pass.

Relation to TiPToP. Our stack differs from TiPToP [59] in both components and purpose. TiPToP pairs Gemini Robotics-ER 1.5 with SAM-2, and feeds the resulting meshes and symbolic goals to a TAMP planner. We instead use Gemini Robotics-ER 1.6, whose spatial and physical reasoning improves over 1.5 [62, 63]. We also replace SAM-2 with SAM 3, whose concept-promptable segmentation grounds open-vocabulary objects more reliably [64]. More importantly, our added annotations let $\hat{\mathcal{X}}$ compile directly into Stage and Composition Specifications for contact-rich behavior composition. The same representation in TiPToP instead feeds a motion planner.

Limitations. The stack builds the Scene Summary once from the initial views, so errors in the estimated poses or frames propagate to every stage. The reactive behavior corrects only contact-observable deviations on its owned axes, not global perception errors. High-precision frames therefore still rely on calibration or object-centric templates rather than purely open-vocabulary estimation. Convex-hull completion yields coarse geometry, sufficient for collision checking but not for fine reasoning on concave parts. Finally, the semantic branch depends on a proprietary model (Gemini Robotics-ER 1.6) served through an API, which constrains reproducibility, offline use, and latency.

11 Sensing hardware and calibration

The sensing stack consists of two calibrated RealSense D415 cameras mounted at fixed external viewpoints and a GelSight Mini tactile sensor on one of the two Franka end-effectors. Camera extrinsics are obtained by an ArUco-board hand-eye calibration repeated whenever the workspace is reconfigured. Object pose estimates that feed the semantic and predictive behaviors are produced by an upstream perception module adapted from prior work; downstream behaviors treat these poses as part of the externally provided Scene Summary $\hat{\mathcal{X}}_t$, and the composition interface is agnostic to the specific pose source. The tactile stream is read at 25 Hz (the GelSight Mini readout rate), and the surface normal map used by the reactive behavior is computed per frame via Poisson integration on the raw gel image. The reactive behavior consumes only the tactile stream and does not depend on the external cameras during contact, which is what allows the corrective loop to continue when an object is occluded by the gripper.

12 Visual Trajectory Generation

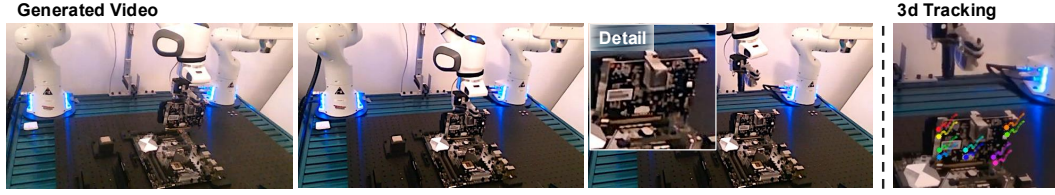


Figure 6: Visual Trajectory Generation and Keypoint Tracking. The left part illustrates the video frames generated by the world model (e.g., GPU insertion), while the right figure depicts the extracted 3D keypoint tracks. These imagined futures provide a high-fidelity motion prior for deriving smooth, object-centric trajectories.